

CS-1201 Object-Oriented Programming

Copy Constructor

Arbish Akram

Department of Computer Science
Government College University

Copying Objects: Two Situations

- Like ordinary variables (but unlike arrays), objects can be copied with the `=` operator.
- By *default*, every member of the source is copied into the matching member of the destination (memberwise copy).
- But `=` appears in two different situations:
 - Assignment: both objects already exist
`box2 = box1;`
 - Initialization: a new object is being created from an existing one
`Rectangle box2 = box1;`
- Creating a new object always runs a constructor. So which constructor runs in the second case?

Example: Rectangle

```
1  class Rectangle {
2      private:
3          double width, length;
4      public:
5          Rectangle(double w, double l) : width(w), length(l) {}
6          double getWidth() const { return width; }
7          double getLength() const { return length; }
8  };
9  int main() {
10     Rectangle box1(10.0, 10.0);
11     Rectangle box2 = box1;    // initialization: new object from box1
12     cout << box2.getWidth() << " " << box2.getLength(); // 10 10
13 }
```

- box2 is a new object, yet we never wrote code to copy box1 into it. Something did it for us.
- It works because Rectangle owns only plain doubles.

Where the Copy Constructor Fits

- Default Constructor: takes no parameters. The compiler provides one if you define no constructors.
- Parameterized Constructor: takes arguments to initialize an object with specific values.
- Copy Constructor: creates a new object as a copy of an existing object. *This is the one that ran for `Rectangle box2 = box1;`*
- Every class has a copy constructor, either
 - the compiler-generated one (you get it for free), or
 - a user-defined one that you write yourself.

When Is the Copy Constructor Called

- When a new object is initialized from another of the same type:

`MyClass obj2 = obj1;    or    MyClass obj2(obj1);`

- When an object is passed by value to a function.
- When an object is returned by value from a function.
- When an object is thrown or caught by value in exceptions.

Not when both objects already exist: `obj2 = obj1;` is assignment, not construction.

The Default Copy Constructor

- Provided automatically by the compiler.
- Performs a *shallow copy*: each member is copied value by value, exactly the memberwise copy we saw with Rectangle.
- Perfectly fine for `int`, `double`, and `string`: there is nothing to share.

```
1  class MyClass {
2      public:
3          int value;
4  };
5
6  MyClass obj1;
7  obj1.value = 10;
8  MyClass obj2 = obj1; // Default copy constructor is called
```

Example: StudentTestScores

```
1  const double DEFAULT_SCORE = 0.0;
2  class StudentTestScores {
3      private:
4          string studentName;
5          double* testScores;    // points to a dynamically allocated array
6          int numTestScores;
7
8          void createTestScoresArray(int size) {
9              numTestScores = size;
10             testScores = new double[size];
11             for (int i = 0; i < size; i++)
12                 testScores[i] = DEFAULT_SCORE;
13         }
14     public:
15         StudentTestScores(string name, int numScores) {
16             studentName = name;
17             createTestScoresArray(numScores);
18         }
19         ~StudentTestScores() { delete[] testScores; }
```

Example: StudentTestScores

```
1 // (other getters and setters omitted)
2 void setTestScore(double score, int index) {
3     testScores[index] = score;
4 }
5 double getTestScore(int index) const {
6     return testScores[index];
7 }
8 };
```

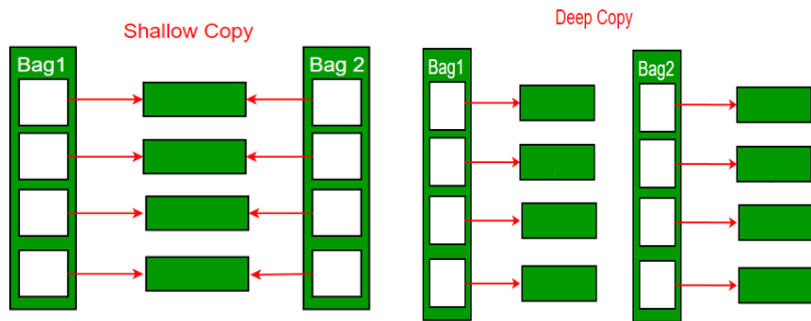
- The constructor allocates an array with `new`; the destructor frees it with `delete[]`.
- So each object owns a chunk of heap memory, and `testScores` is just a pointer to it.
- We have not written a copy constructor. What happens if we copy one of these objects?

Example: StudentTestScores

```
1  int main() {
2      StudentTestScores s1("Maria Jones Tucker", 5);
3      s1.setTestScore(90.0, 0);
4
5      StudentTestScores s2 = s1;    // default copy ctor: SHALLOW
6      s2.setTestScore(50.0, 0);    // change s2's first score...
7
8      cout << s1.getTestScore(0);  // prints 50, not 90!
9  }    // s2 destroyed: delete[] testScores
10     // s1 destroyed: delete[] the SAME address again -> undefined behavior
```

- The default copy constructor copied the pointer's value (an address), not the array it points to.
- s1 and s2 now share one array.

Shallow vs Deep Copy



- Shallow copy (what just happened): both pointers hold the same address. One locker, two keys.
- Deep copy (what we need): the target is duplicated too. Each object gets its own locker.

Why Shallow Copy Fails Here

Sharing one resource between two owners causes two bugs:

- Silent data corruption: changing `s2` changes `s1`.
- *Double deletion*: both destructors call `delete[]` on the same address, which is undefined behavior.

This applies whenever a class owns:

- pointers to dynamically allocated memory
- file handles or other system resources

The fix: write our own copy constructor. First, the general syntax, then we'll apply it to `StudentTestScores`.

Writing Our Own Copy Constructor: Syntax

```
ClassName(const ClassName &other) {  
    // Copy member variables from 'other'  
}
```

- `const ClassName &other:`
 - Passed by reference: passing by value would itself need a copy constructor call, an infinite loop.
 - Marked `const`: a copy constructor has no reason to modify the object it copies from.
- The body decides *how* to copy each member.

Example: Student

```
1  class Student {
2      int rollNumber;
3      string Name;
4      public:
5          Student(int number, string name) {
6              rollNumber = number;
7              Name = name;
8          }
9          // Copy constructor
10         Student(const Student &t) {
11             rollNumber = t.rollNumber;
12             Name = t.Name;
13             cout << "Copy Constructor Called" << endl;
14         }
15         void display() { cout << rollNumber << "\t" << Name << endl; }
16     };
```

Example: Student

```
1  int main() {  
2      Student s1(501, "Ali");  
3      s1.display();  
4  
5      Student s2(s1);    // copy constructor called  
6      s2.display();  
7  }
```

- The message prints once, at `Student s2(s1);`.
- `Student` has no pointer members, so this copy constructor does exactly what the default one would.
- Now let's apply this syntax to fix `StudentTestScores`.

Example: The Deep-Copy Fix

```
1 // Deep-copy constructor for StudentTestScores
2 StudentTestScores(const StudentTestScores &other) {
3     studentName    = other.studentName;    // string copies itself safely
4     numTestScores  = other.numTestScores;
5     // allocate NEW memory -- do not reuse other's pointer
6     testScores = new double[numTestScores];
7     // copy the actual values, not the address
8     for (int i = 0; i < numTestScores; i++)
9         testScores[i] = other.testScores[i];
10 }
```

- Now `StudentTestScores s2 = s1;` gives `s2` its own array with the same values.
- Re-run the earlier example: `s1` still prints 90, and each destructor frees a different array.

Summary

- A new object created from an existing one runs the copy constructor. With two existing objects, = is assignment instead.
- The compiler-generated version copies members value by value (shallow copy). That is correct for plain data.
- For a pointer member it copies the address, not what it points to. Two objects then share one resource, risking corruption and double deletion.
- A deep-copy constructor allocates new memory and copies the contents, giving each object independent ownership.
- Rule of thumb: if your class owns a pointer, a file handle, or any dynamically managed resource, write your own copy constructor. If it holds only plain values, the default is fine.